

Database Management Systems: Assignment-9

Name: Suraj Shah

Admission Number: I24AI012

Q 1. Consider the following schema:

Sailors (sid, sname, rating, age)

Boats (bid, bname, color)

Reserves (sid, bid, day(date))

Write subquery statement for the following queries. The bold column names are indicating primary key.

☐	☐	 sid int(11)		sname varchar(30)		rating int(11)		age decimal(4,1)	
☐	>	1		Bob		7		25.5	
☐	>	2		Alice		8		30.0	
☐	>	3		John		5		22.0	
☐	>	4		David		9		35.0	
☐	>	5		Eva		7		19.0	
☐	>	6		Chris		5		22.0	

☐	☐	 bid int(11)		bname varchar(30)		color varchar(15)	
☐	>	101		Ocean Queen		red	
☐	>	102		Sea Rider		blue	
☐	>	103		Wave King		green	
☐	>	104		Sun Chaser		red	
☐	>	105		Aqua Dream		yellow	

		sid int(11)	bid int(11)	day date
	>	1	101	2024-05-10
	>	1	103	2024-05-10
	>	2	102	2024-05-11
	>	3	104	2024-05-12
	>	4	103	2024-05-13
	>	5	101	2024-05-14
	>	6	103	2024-05-15
	>	6	105	2024-05-15

- a) Find all information of sailors who have reserved boat number 101.

Query:

```
SELECT * FROM Sailors WHERE sid IN (SELECT sid FROM Reserves WHERE bid=101);
```

Output:

	sid int	sname varchar	rating int	age decimal
>	1	Bob	7	25.5
>	5	Eva	7	19.0

- b) Find the name of boat reserved by Bob.

Query:

```
SELECT bname FROM Boats WHERE bid IN (SELECT bid FROM Reserves WHERE sid =
(SELECT sid FROM Sailors WHERE sname='Bob'));
```

Output:

bname varchar
> Ocean Queen
> Wave King

- c) Find the names of sailors who have reserved a red boat, and list in the order of age.

Query:

```
SELECT sname FROM Sailors WHERE sid IN (SELECT sid FROM Reserves WHERE bid IN
(SELECT bid FROM Boats WHERE color='Red')) ORDER BY age;
```

Output:

Q	sname varchar	⬆️⬇️⬆️
>	Eva	
>	John	
>	Bob	

- d) Find the names of sailors who have reserved at least one boat.

Query:

```
SELECT sname from Sailors WHERE sid IN (SELECT DISTINCT sid FROM Reserves);
```

Output:

Q	sname varchar	⬆️⬇️⬆️
>	Bob	
>	Alice	
>	John	
>	David	
>	Eva	
>	Chris	

- e) Find the ids and names of sailors who have reserved two different boats on the same day.

Query:

```
SELECT sid, sname FROM Sailors WHERE sid IN (SELECT sid from Reserves GROUP BY
sid, day HAVING COUNT(DISTINCT bid) > 1);
```

Output:

Q	sid int	⬆️⬇️⬆️	sname varchar	⬆️⬇️⬆️
>	1		Bob	
>	6		Chris	

- f) Find the ids of sailors who have reserved a red boat or a green boat.

Query:

```
SELECT DISTINCT sid FROM Reserves WHERE bid IN (SELECT bid FROM Boats WHERE color IN ('red', 'green'));
```

Output:

	sid int
>	1
>	5
>	4
>	6
>	3

- g) Find the name and the age of the youngest sailor.

Query:

```
SELECT sname, age FROM Sailors WHERE age = (SELECT MIN(age) FROM Sailors);
```

Output:

	sname varchar	age decimal
>	Eva	19.0

- h) Count the number of different sailor names.

Query:

```
SELECT COUNT(DISTINCT sname) AS total_names from Sailors;
```

Output:

	total_names
>	6

- i) Find the average age of sailors for each rating level.

Query:

```
SELECT rating, AVG(age) AS avg_age FROM Sailors GROUP BY rating;
```

Output:

	Q	rating int(11)	avg_age
	>	5	22.00000
	>	7	22.25000
	>	8	30.00000
	>	9	35.00000

- j) Find the average age of sailors for each rating level that has at least two sailors.

Query:

```
SELECT rating, AVG(age) AS avg_age FROM Sailors GROUP BY rating HAVING  
COUNT(*) >= 2;
```

Output:

	Q	rating int(11)	avg_age
	>	5	22.00000
	>	7	22.25000

Q 2. Consider the following database for a banking enterprise.

BRANCH (branch_name:string , branch_city:string , assets:real)

ACCOUNT (accno:int , branch_name:string , balance:real)

DEPOSITOR (customer_name:string , accno:int)

CUSTOMER (customer_name:string , customer_street:string , customer_city:string)

LOAN (loan_number:int , branch_name:string , amount:real)

BORROWER (customer_name:string , loan_number:int)

a) Enter at least five tuples for each relation.

Query:

INSERT INTO BRANCH VALUES

('Main', 'Mumbai', 1000000.00),
('Town', 'Mumbai', 750000.00),
('Park', 'Delhi', 600000.00),
('Central', 'Pune', 500000.00),
('South', 'Chennai', 550000.00);

Output:

		 branch_name varchar(30)  	branch_city varchar(30)  	assets decimal(15,2)  
	>	Central	Pune	500000.00
	>	Main	Mumbai	1000000.00
	>	Park	Delhi	600000.00
	>	South	Chennai	550000.00
	>	Town	Mumbai	750000.00

Query:

```
INSERT INTO ACCOUNT VALUES
(101, 'Main', 50000.00),
(102, 'Main', 30000.00),
(103, 'Town', 25000.00),
(104, 'Park', 45000.00),
(105, 'Central', 60000.00),
(106, 'South', 20000.00),
(107, 'Main', 70000.00),
(108, 'Town', 15000.00);
```

Output:

		 accno int(11)  	 branch_name varchar(30)  	balance decimal(15,2)  
	>	101	Main	50000.00
	>	102	Main	30000.00
	>	103	Town	25000.00
	>	104	Park	45000.00
	>	105	Central	60000.00
	>	106	South	20000.00
	>	107	Main	70000.00
	>	108	Town	15000.00

Query:

```
INSERT INTO CUSTOMER VALUES
('Rahul', 'MG Road', 'Mumbai'),
('Sneha', 'Hill Street', 'Delhi'),
('Amit', 'Park Lane', 'Pune'),
('Priya', 'Marine Drive', 'Mumbai'),
('Ravi', 'Anna Nagar', 'Chennai'),
('Neha', 'Bandra', 'Mumbai');
```

Output:

		customer_name varchar(30)	accno int(11)
☐	>	Rahul	101
☐	>	Rahul	102
☐	>	Priya	103
☐	>	Amit	104
☐	>	Ravi	105
☐	>	Neha	107
☐	>	Sneha	106
☐	>	Priya	108

Query:

```
INSERT INTO DEPOSITOR VALUES
('Rahul', 101),
('Rahul', 102),
('Priya', 103),
('Amit', 104),
('Ravi', 105),
('Neha', 107),
('Sneha', 106),
('Priya', 108);
```

Output:

		customer_name varchar(30)	customer_street varchar(50)	customer_city varchar(30)
☐	>	Amit	Park Lane	Pune
☐	>	Neha	Bandra	Mumbai
☐	>	Priya	Marine Drive	Mumbai
☐	>	Rahul	MG Road	Mumbai
☐	>	Ravi	Anna Nagar	Chennai
☐	>	Sneha	Hill Street	Delhi

Query:

```
INSERT INTO LOAN VALUES
(201, 'Main', 100000.00),
(202, 'Town', 50000.00),
(203, 'Park', 120000.00),
(204, 'Central', 80000.00),
(205, 'South', 60000.00);
```

Output:

		loan_number int(11)	branch_name varchar(30)	amount decimal(15,2)
	>	201	Main	100000.00
	>	202	Town	50000.00
	>	203	Park	120000.00
	>	204	Central	80000.00
	>	205	South	60000.00

Query:

```
INSERT INTO BORROWER VALUES
('Rahul', 201),
('Amit', 203),
('Priya', 202),
('Ravi', 205),
('Neha', 204);
```

Output:

		customer_name varchar(30)	loan_number int(11)
	>	Rahul	201
	>	Amit	203
	>	Priya	202
	>	Ravi	205
	>	Neha	204

- b) Find all the customers who have at least two accounts at the main branch.

Query:

```
SELECT customer_name FROM Depositor WHERE accno IN (SELECT accno FROM
Account WHERE branch_name = 'Main') GROUP BY customer_name HAVING COUNT(accno)
>= 2;
```

Output:

	customer_name varchar
>	Rahul

- c) Find all the customers who have an account at all the branches located in a specific city.

Query:

```
SELECT D.customer_name FROM Depositor D WHERE NOT EXISTS (SELECT  
B.branch_name FROM Branch B WHERE B.branch_city = 'Mumbai' AND B.branch_name NOT  
IN (SELECT A.branch_name FROM Account A where A.accno IN (SELECT D2.accno FROM  
Depositor D2 WHERE D2.customer_name = D.customer_name)));
```

Output:

customer_name
varchar

- d) Demonstrate how you delete all account tuples at every branch located in a specific city.

Query:

```
DELETE FROM Depositor WHERE accno IN (SELECT accno FROM Account WHERE  
branch_name IN (SELECT branch_name FROM Branch WHERE branch_city = 'Mumbai'));  
DELETE FROM Account WHERE branch_name IN (SELECT branch_name FROM Branch  
WHERE branch_city = 'Mumbai');
```

Output:

	accno	branch_name	balance
	int(11)	varchar(30)	decimal(15,2)
>	104	Park	45000.00
>	105	Central	60000.00
>	106	South	20000.00

Q 3. The following are maintained by a book dealer.

AUTHOR(author_id:int , name:string , city:string , country:string)

PUBLISHER(publisher_id:int , name:string , city:string , country:string)

CATALOG(book_id:int , title:string , author_id:int , publisher_id:int , category_id:int,
year:int , price:int)

CATEGORY(category_id:int , description:string)

ORDER_DETAILS(order_no:int , book_id:int , quantity:int)

- a) Create the above tables by properly specifying the primary keys and foreign keys.

Query:

```
CREATE TABLE AUTHOR (author_id INT PRIMARY KEY, name VARCHAR(50), city  
VARCHAR(30), country VARCHAR(30));
```

Output:

Q	Field varchar	Type blob	Null varchar	Key varchar	Default blob	Extra varchar
>	author_id	int(11)	NO	PRI	(NULL)	
>	name	varchar(50)	YES		(NULL)	
>	city	varchar(30)	YES		(NULL)	
>	country	varchar(30)	YES		(NULL)	

Query:

```
CREATE TABLE PUBLISHER (publisher_id INT PRIMARY KEY, name VARCHAR(50), city  
VARCHAR(30), country VARCHAR(30));
```

Output:

Q	Field varchar	Type blob	Null varchar	Key varchar	Default blob	Extra varchar
>	publisher_id	int(11)	NO	PRI	(NULL)	
>	name	varchar(50)	YES		(NULL)	
>	city	varchar(30)	YES		(NULL)	
>	country	varchar(30)	YES		(NULL)	

Query:

```
CREATE TABLE CATEGORY (category_id INT PRIMARY KEY, description VARCHAR(50));
```

Output:

Q	Field varchar	Type blob	Null varchar	Key varchar	Default blob	Extra varchar
>	category_id	int(11)	NO	PRI	(NULL)	
>	description	varchar(50)	YES		(NULL)	

Query:

```
CREATE TABLE CATALOG (book_id INT PRIMARY KEY, title VARCHAR(100), author_id INT, publisher_id INT, category_id INT, year INT, price DECIMAL(10,2), FOREIGN KEY (author_id) REFERENCES AUTHOR(author_id), FOREIGN KEY (publisher_id) REFERENCES PUBLISHER(publisher_id), FOREIGN KEY (category_id) REFERENCES CATEGORY(category_id));
```

Output:

Q	Field varchar	Type blob	Null varchar	Key varchar	Default blob	Extra varchar
>	book_id	int(11)	NO	PRI	(NULL)	
>	title	varchar(100)	YES		(NULL)	
>	author_id	int(11)	YES	MUL	(NULL)	
>	publisher_id	int(11)	YES	MUL	(NULL)	
>	category_id	int(11)	YES	MUL	(NULL)	
>	year	int(11)	YES		(NULL)	
>	price	decimal(10,2)	YES		(NULL)	

Query:

```
CREATE TABLE ORDER_DETAILS (order_no INT, book_id INT, quantity INT, PRIMARY KEY (order_no, book_id), FOREIGN KEY (book_id) REFERENCES CATALOG(book_id));
```

Output:

Q	Field varchar	Type blob	Null varchar	Key varchar	Default blob	Extra varchar
>	order_no	int(11)	NO	PRI	(NULL)	
>	book_id	int(11)	NO	PRI	(NULL)	
>	quantity	int(11)	YES		(NULL)	

b) Enter at least five tuples for each relation.

Query:

```
INSERT INTO AUTHOR VALUES
(1, 'Chetan Bhagat', 'Mumbai', 'India'),
(2, 'J.K. Rowling', 'London', 'UK'),
(3, 'George R.R. Martin', 'New Jersey', 'USA'),
(4, 'Ruskin Bond', 'Dehradun', 'India'),
(5, 'Dan Brown', 'New York', 'USA');
```

Output:

		 author_id int(11)	 name varchar(50)	 city varchar(30)	 country varchar(30)
	>	1	Chetan Bhagat	Mumbai	India
	>	2	J.K. Rowling	London	UK
	>	3	George R.R. Martin	New Jersey	USA
	>	4	Ruskin Bond	Dehradun	India
	>	5	Dan Brown	New York	USA

Query:

```
INSERT INTO PUBLISHER VALUES
(1, 'Penguin Books', 'Delhi', 'India'),
(2, 'Bloomsbury', 'London', 'UK'),
(3, 'HarperCollins', 'New York', 'USA'),
(4, 'Rupa Publications', 'Mumbai', 'India'),
(5, 'Random House', 'Boston', 'USA');
```

Output:

		 publisher_id int(11)	 name varchar(50)	 city varchar(30)	 country varchar(30)
	>	1	Penguin Books	Delhi	India
	>	2	Bloomsbury	London	UK
	>	3	HarperCollins	New York	USA
	>	4	Rupa Publications	Mumbai	India
	>	5	Random House	Boston	USA

Query:

```
INSERT INTO CATEGORY VALUES
```

```
(1, 'Fiction'),
(2, 'Fantasy'),
(3, 'Mystery'),
(4, 'Thriller'),
(5, 'Romance');
```

Output:

		category_id int(11)		description varchar(50)	
	>	1		Fiction	
	>	2		Fantasy	
	>	3		Mystery	
	>	4		Thriller	
	>	5		Romance	

Query:

```
INSERT INTO CATALOG VALUES
```

```
(101, 'Five Point Someone', 1, 4, 1, 2004, 300),
(102, '2 States', 1, 4, 5, 2009, 350),
(103, 'Harry Potter', 2, 2, 2, 2001, 500),
(104, 'Game of Thrones', 3, 3, 2, 2005, 800),
(105, 'Angels and Demons', 5, 5, 4, 2003, 600),
(106, 'Room on the Roof', 4, 1, 1, 1956, 250),
(107, 'Half Girlfriend', 1, 4, 1, 2014, 400),
(108, 'The Da Vinci Code', 5, 5, 3, 2004, 700);
```

Output:

		book_id int(11)		title varchar(100)		author_id int(11)		publisher_id int(11)		category_id int(11)		year int(11)		price decimal(10,2)	
	>	101		Five Point Someone		1		4		1		2004		300.00	
	>	102		2 States		1		4		5		2009		350.00	
	>	103		Harry Potter		2		2		2		2001		500.00	
	>	104		Game of Thrones		3		3		2		2005		800.00	
	>	105		Angels and Demons		5		5		4		2003		600.00	
	>	106		Room on the Roof		4		1		1		1956		250.00	
	>	107		Half Girlfriend		1		4		1		2014		400.00	
	>	108		The Da Vinci Code		5		5		3		2004		700.00	

Query:

```
INSERT INTO ORDER_DETAILS VALUES
(1001, 101, 20),
(1002, 102, 15),
(1003, 103, 30),
(1004, 104, 25),
(1005, 105, 40),
(1006, 107, 10),
(1007, 108, 50);
```

Output:

		order_no int(11)	book_id int(11)	quantity int(11)
	>	1001	101	20
	>	1002	102	15
	>	1003	103	30
	>	1004	104	25
	>	1005	105	40
	>	1006	107	10
	>	1007	108	50

- c) Give the details of the authors who have 2 or more books in the catalog and the price of the books is greater than the average price of the books in the catalog and the year of publication is after 2000.

Query:

```
SELECT DISTINCT A.author_id, A.name, A.city, A.country FROM Author A WHERE
A.author_id IN (SELECT author_id FROM Catalog WHERE price > (SELECT AVG(price) FROM
Catalog) AND year > 2000 GROUP BY author_id HAVING COUNT(book_id) >= 2);
```

Output:

	author_id int	name varchar	city varchar	country varchar
>	5	Dan Brown	New York	USA

- d) Find the author of the book that has maximum sales.

Query:

```
SELECT A.name, A.city, A.country FROM Author A JOIN Catalog C ON A.author_id =  
C.author_id JOIN (SELECT book_id FROM Order_details GROUP BY book_id HAVING  
SUM(quantity) = (SELECT MAX(total_sales) FROM (SELECT SUM(quantity) AS total_sales  
FROM Order_details GROUP BY book_id) AS t)) AS TopBooks ON c.book_id =  
TopBooks.book_id;
```

Output:

	name varchar	city varchar	country varchar
>	Dan Brown	New York	USA

- e) Demonstrate how you increase the price of books published by a specific publisher by 10%.

Query:

```
UPDATE Catalog SET price = price * 1.1 WHERE publisher_id = (SELECT publisher_id  
FROM Publisher WHERE name = 'Rupa Publications');
```

Output:

	book_id int(11)	title varchar(100)	author_id int(11)	publisher_id int(11)	category_id int(11)	year int(11)	price decimal(10,2)
>	101	Five Point Someone	1	4	1	2004	330.00
>	102	2 States	1	4	5	2009	385.00
>	103	Harry Potter	2	2	2	2001	500.00
>	104	Game of Thrones	3	3	2	2005	800.00
>	105	Angels and Demons	5	5	4	2003	600.00
>	106	Room on the Roof	4	1	1	1956	250.00
>	107	Half Girlfriend	1	4	1	2014	440.00
>	108	The Da Vinci Code	5	5	3	2004	700.00